

CS-E4650 Methods of Data Mining, Homework N

Student Name (student id 000000) and Student Name (student id 000000)

1 Methods

We used Python with NumPy for the computations and Matplotlib for the figures. The data file contains 1000 three-dimensional vectors with all values in $[0, 1]$ (checked in the code). For the entropies, a subset of k dimensions was divided into $m = 64$ equal grid cells by splitting each of its dimensions into $\varphi = 64^{1/k}$ equal ranges (4, 8 and 64 ranges for $k = 3, 2$ and 1); a value x falls into range $\lfloor \varphi x \rfloor$, and $x = 1$ into the last range. The greedy searches were simulated with the seven resulting entropies.

2 Histograms

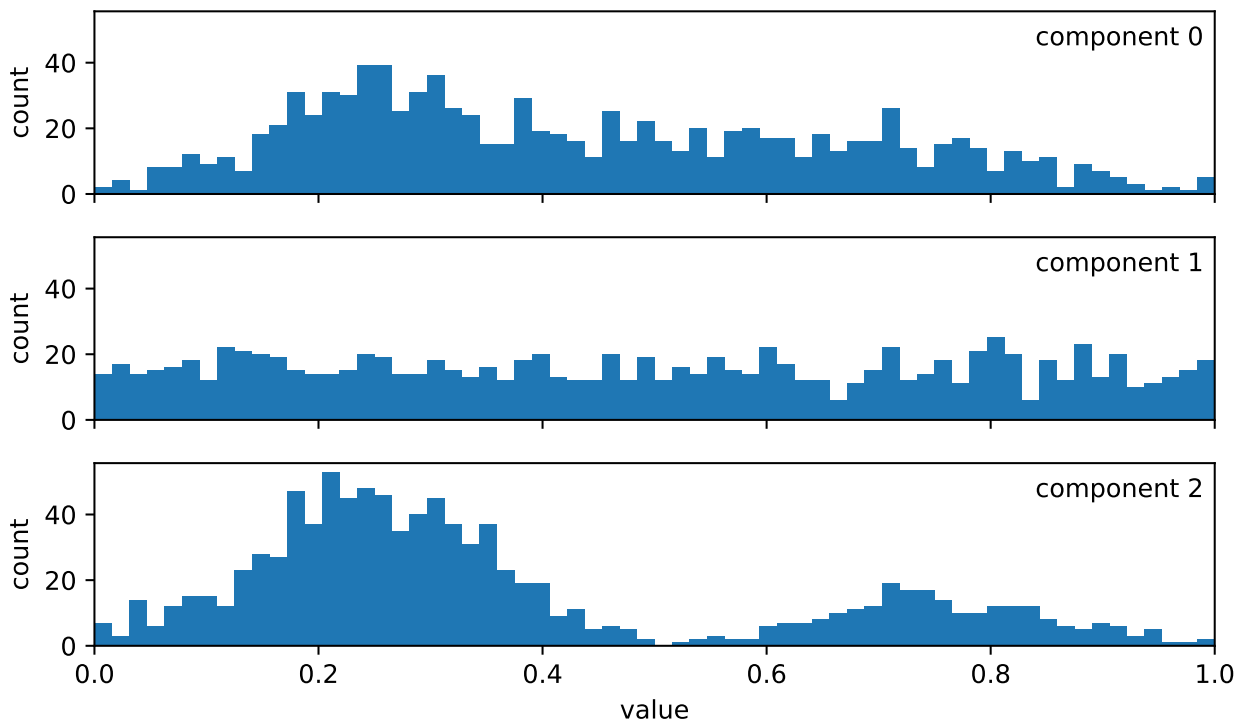


Figure 1: Histograms of the components (64 bins).

Component 0 (Figure 1) has a peak at about 0.25 and a broad shoulder up to about 0.9. It is clearly non-uniform and suggests overlapping groups, but there is no gap that would separate them, so on its own it shows only a weak clustering tendency. Component 1 is flat, with only random fluctuations, and shows no clustering tendency. Component 2 is

clearly bimodal: about three quarters of the points lie around 0.25 and the rest around 0.75, separated by an almost empty gap at 0.5. It alone shows a strong clustering tendency with two groups. Judged by the histograms, component 2 contributes most to the clustering tendency, component 0 some and component 1 nothing.

3 Scatter plots

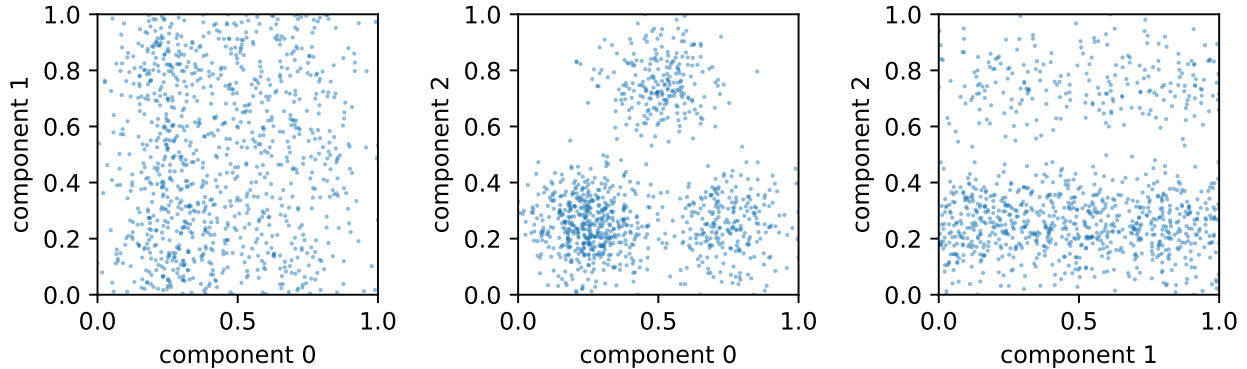


Figure 2: Scatter plots of the component pairs (equal aspect ratio).

Components 0 and 1 (Figure 2) show no groups: the density varies only along component 0, in the same way for every value of component 1, so the two components appear independent and their combination shows no clusters.

Components 0 and 2 show three compact, well-separated clusters with roughly half, a quarter and a quarter of the points, centred at about $(0.25, 0.25)$, $(0.75, 0.25)$ and $(0.5, 0.75)$. The components are strongly but nonlinearly dependent: when component 2 is low, component 0 is around 0.25 or 0.75, and when component 2 is high, around 0.5. Neither component alone shows all three clusters: component 2 merges the two lower clusters, and in component 0 the upper cluster fills the gap between the lower two. The pair therefore has a clearly stronger clustering tendency than either component.

Components 1 and 2 show two horizontal bands (component 2 ≈ 0.25 and ≈ 0.75) spread evenly over component 1, i.e. only the bimodality of component 2 and no dependence on component 1.

Component 1 is thus uniform noise, independent of the other components. Including it stretches each cluster over the whole component-1 axis and dilutes the clustering tendency; the cluster structure lies in the subspace of components 0 and 2.

4 Entropies

With p_i the fraction of points in cell i , the probability-based entropy (Aggarwal, Section 6.2.1.3, eq. (6.2)) is

$$E = - \sum_{i=1}^m [p_i \ln p_i + (1 - p_i) \ln(1 - p_i)],$$

i.e. each cell is treated as an independent binary variable that is occupied with probability p_i and empty with $1 - p_i$; $0 \ln 0 = 0$. Since $m = 64$ for every subset, all values share the same range: from 0 (all points in one cell) to $E_{\max} = \ln 64 - 63 \ln \frac{63}{64} \approx 5.151$ (all $p_i = 1/64$).

Table 1: Probability-based entropies of all combinations of dimensions ($m = 64$).

Dimensions	$\{0, 1, 2\}$	$\{0, 1\}$	$\{0, 2\}$	$\{1, 2\}$	$\{0\}$	$\{1\}$	$\{2\}$
E	4.794	4.998	4.482	4.792	4.954	5.118	4.761

All values in Table 1 lie in a narrow range, 4.482–5.118 (87–99% of $E_{\max}$), but the differences are consistent. $E_{\{0,2\}}$ is clearly the lowest, so $\{0, 2\}$ has the strongest clustering tendency. $E_{\{1\}}$ is within 1% of $E_{\max}$ (practically uniform), and adding component 1 always increases the entropy: by 0.044 for $\{0\}$, 0.031 for $\{2\}$ and 0.312 for $\{0, 2\}$. With m fixed, an extra dimension adds no cells but lowers the resolution of the others, and a uniform component only spreads the points evenly. Component 2 has the lowest single entropy (4.761), and adding it always decreases the entropy: by 0.472 for $\{0\}$, 0.326 for $\{1\}$ and 0.204 for $\{0, 1\}$. It is the strongest single contributor. Component 0 alone is moderate (4.954) but lowers the entropy of $\{2\}$ by 0.279, so it contributes mainly together with component 2. In the coarse $4 \times 4 \times 4$ grid, however, adding it to $\{1, 2\}$ does not lower the entropy (+0.002, see Section 7).

5 Greedy backward selection

Starting from all dimensions, each stage drops the worst feature, i.e. the one whose removal gives the lowest entropy, until every removal would increase the entropy (Table 2).

Table 2: Stages of greedy backward selection (lowest candidate in bold).

Stage	Current set (E)	Candidates (E)	Decision
1	$\{0, 1, 2\}$ (4.794)	$\{1, 2\}$: 4.792, $\{0, 2\}$: 4.482 , $\{0, 1\}$: 4.998	drop 1
2	$\{0, 2\}$ (4.482)	$\{2\}$: 4.761 , $\{0\}$: 4.954	stop

In stage 1, dropping component 1 gives by far the largest decrease (-0.312), whereas dropping component 0 barely changes the entropy (-0.002) and dropping component 2 increases it ($+0.204$). In stage 2, both removals increase the entropy ($+0.279$ and $+0.472$), so the search stops at $\{0, 2\}$ with $E = 4.482$.

6 Greedy forward selection

Starting from the empty set, each stage adds the best feature, i.e. the one that gives the lowest entropy, until every addition would increase the entropy (Table 3). The empty set has no entropy on a 64-cell grid, so the first stage takes the single dimension with the lowest entropy.

In stage 1, component 2 has the lowest entropy. In stage 2, adding component 0 decreases the entropy by 0.279, whereas adding component 1 increases it by 0.031. In stage 3, adding component 1 would increase the entropy by 0.312, so the search stops at $\{0, 2\}$, the same result as backward selection.

Table 3: Stages of greedy forward selection (lowest candidate in bold).

Stage	Current set (E)	Candidates (E)	Decision
1	$\emptyset$	$\{0\}$: 4.954, $\{1\}$: 5.118, $\{2\}$: 4.761	add 2
2	$\{2\}$ (4.761)	$\{0, 2\}$: 4.482 , $\{1, 2\}$: 4.792	add 0
3	$\{0, 2\}$ (4.482)	$\{0, 1, 2\}$: 4.794	stop

7 Analysis

The three approaches agree. Visually, the clusters lie in the plane of components 0 and 2 and component 1 is uniform noise. The entropies confirm this: $\{0, 2\}$ has the lowest entropy, $\{1\}$ is close to $E_{\max}$, and adding component 1 always increases the entropy. Both greedy searches end at $\{0, 2\}$, which is also the minimum of all seven values, i.e. the result of an exhaustive search, as expected from the plots. They agree because the data are easy for greedy search: component 1 is harmful in every combination, so backward selection drops it first, and components 0 and 2 lower the entropy both alone and together, so forward selection adds them.

Two limitations are visible. First, with m fixed, the resolution per dimension falls with the dimensionality (64, 8 and 4 ranges). In the $4 \times 4 \times 4$ grid, the range boundaries 0.25, 0.5 and 0.75 cut through the cluster centres, so $E_{\{0,1,2\}}$ is not lower than $E_{\{1,2\}}$ although the full data contain all three clusters. With less clear data, a decision made on such a coarse grid could drop a useful component, and forward selection could miss components that are informative only jointly. Second, entropy measures non-uniformity rather than cluster separation: component 0 has a clearly lower entropy than the uniform component 1 although its histogram shows no separated groups. The visual inspection is needed to interpret the entropies.

8 Why 64 bins?

$64 = 4^3 = 8^2 = 64^1$, so uniform grids with equally many ranges per dimension give exactly $m = 64$ cells for every one-, two- and three-dimensional subset. The entropy depends on m (its maximum is determined by m), so m should be the same for all compared subsets. In general this is only approximately possible, with $\lceil m^{1/k} \rceil$ ranges per dimension. Equal m matters especially for the greedy searches, whose stopping decisions compare subsets of different dimensionality. In addition, $1000/64 \approx 15.6$ points per cell suffice to estimate the p_i , and because the range widths $1/4$, $1/8$ and $1/64$ are powers of two, the bin edges are exact in binary floating point.

Other values with the same property are the sixth powers 1, 64, 729, 4096, ... With $m = 1$ every entropy is 0. $m = 729 = 9^3 = 27^2$ leaves only 1.4 points per cell (even for uniform data about a quarter of the cells would be empty), and $m = 4096$ exceeds the number of points. So 64 is the only practical choice for these data. If only pairs were compared, any square (e.g. 16 or 100) would do; for triples, any cube (e.g. 27 or 125).

With d -dimensional data, an equal m for all subset sizes $1, \dots, d$ requires $m = q^{\text{lcm}(1, \dots, d)}$, i.e. at least $2^{12} = 4096$ cells for $d = 4$ and 2^{60} for $d = 5$, which is impossible. Moreover, a k -dimensional grid with at least two ranges per dimension has at least 2^k cells, so the number of points needed grows exponentially and most cells become empty. In practice,

m is chosen from the number of points (e.g. at least 5–10 points per cell), each dimension gets $\lceil m^{1/k} \rceil$ ranges, entropies are compared relative to their maximum, and the search is limited to low-dimensional subsets. The alternative is to compute the entropy from the one-dimensional distribution of pairwise distances, whose number of bins does not depend on the dimensionality.

9 Conclusions

The data have a clear clustering tendency only in the subspace of components 0 and 2, which contains three well-separated clusters. Component 1 is uniform, independent noise, component 2 is the most informative single component, and component 0 matters mainly together with component 2. The probability-based entropies and both greedy searches identify the same optimal subset, $\{0, 2\}$. Grid entropies are, however, sensitive to the grid resolution and measure non-uniformity rather than cluster separation, so they should be interpreted together with the visual inspection.

10 Appendix – Code

```

"""CS-E4650 homework: clustering tendency and entropy-based measures.

Writes figures/histograms.pdf and figures/scatter.pdf and prints the
entropies and the stages of the greedy searches.
"""

from itertools import combinations
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt # noqa: E402
import numpy as np # noqa: E402

ROOT = Path(__file__).resolve().parents[1]
X = np.loadtxt(ROOT / "data/clustering-tendency.csv", delimiter=",")
assert X.shape == (1000, 3) and X.min() >= 0 and X.max() <= 1
DIMS = (0, 1, 2)
M = 64 # grid cells in every entropy calculation

# histograms, 64 bins
fig, axes = plt.subplots(3, 1, figsize=(7, 4.2), sharex=True, sharey=True)
for j, ax in enumerate(axes):
    ax.hist(X[:, j], bins=64, range=(0, 1))
    ax.text(0.99, 0.92, f"component {j}", transform=ax.transAxes, ha="right", va="top")
    ax.set(xlim=(0, 1), ylabel="count")
axes[-1].set_xlabel("value")
fig.tight_layout()
fig.savefig(ROOT / "figures/histograms.pdf", bbox_inches="tight")

# scatter plots of the pairs, equal aspect ratio
fig, axes = plt.subplots(1, 3, figsize=(7.5, 2.7))
for ax, (i, j) in zip(axes, combinations(DIMS, 2)):
    ax.scatter(X[:, i], X[:, j], s=3, alpha=0.5, linewidths=0)

```

```

    ax.set(xlim=(0, 1), ylim=(0, 1), aspect="equal",
           xlabel=f"component {i}", ylabel=f"component {j}")
fig.tight_layout()
fig.savefig(ROOT / "figures/scatter.pdf", bbox_inches="tight")

def plogp(q):
    """Sum of  $q \ln q$  with  $0 \ln 0 = 0$ ."""
    q = q[q > 0]
    return np.sum(q * np.log(q))

def prob_entropy(p):
    """Aggarwal eq. (6.2):  $E = -\sum_i [p_i \ln p_i + (1 - p_i) \ln(1 - p_i)]$ ."""
    return float(-(plogp(p) + plogp(1 - p)))

def entropy(dims):
    """E of the dimensions dims on a uniform grid of M cells in  $[0,1]^k$ ."""
    k = len(dims)
    phi = round(M ** (1 / k)) # ranges per dimension: 4, 8 or 64
    idx = np.minimum((X[:, dims] * phi).astype(int), phi - 1) #  $x = 1 \rightarrow$  last range
    counts = np.bincount(np.ravel_multi_index(idx.T, (phi,) * k), minlength=M)
    return prob_entropy(counts / len(X))

subsets = [s for k in (3, 2, 1) for s in combinations(DIMS, k)]
E = {s: entropy(list(s)) for s in subsets}
print(f"E_max = {prob_entropy(np.full(M, 1 / M)):.3f}")
for s in subsets:
    print(s, f"{E[s]:.3f}")

def backward():
    """Drop the worst feature (lowest E after removal) while E decreases."""
    cur = DIMS
    while len(cur) > 1:
        cand = [tuple(d for d in cur if d != r) for r in cur]
        print(" ", cur, {c: round(E[c], 3) for c in cand})
        best = min(cand, key=E.get)
        if E[best] >= E[cur]:
            break
        cur = best
    return cur

def forward():
    """Add the best feature (lowest E after addition) while E decreases.
    The empty set has no entropy, so the best single feature starts."""
    cur = ()
    while len(cur) < len(DIMS):
        cand = [tuple(sorted(cur + (a,))) for a in DIMS if a not in cur]
        print(" ", cur, {c: round(E[c], 3) for c in cand})
        best = min(cand, key=E.get)
        if cur and E[best] >= E[cur]:
            break
        cur = best

```

```
return cur
```

```
print("backward selection:")  
print("  result", backward())  
print("forward selection:")  
print("  result", forward())
```